MS2 Summary · WinterLoad: Day-Ahead Forecasting of Swiss Electricity Demand

Björn Pfoster · MLOps HS26 · https://github.com/D4rkMindz/hslu-winterload

1 What I achieved in this milestone

- **Feature pipeline with backfill.** Backfills write hourly load (ENTSO-E) and the weather forecasts for four cities day by day into BigQuery behind Feast, and a rerun skips the days already stored. The store holds load from 2022 and weather from 2024-03-16 to 2026-10-31. Feast on BigQuery replaces the Parquet repository of the proposal, and `OFFLINE_STORAGE=s3` swaps in a local SeaweedFS store. Start at `src/winterload/pipeline/features/`.
- **Weather that existed at 08:00.** Reviewer A and the editors doubted my weather archive, and they were right. My first fix failed too: Open-Meteo published the 00 UTC run at 06:51 UTC, after 08:00 Zurich time in summer. The store now holds the 18 UTC run of D-2. Against ERA5 it costs 0.14 °C of temperature error in February and gains 0.06 °C in July.
- **Cost of the fix.** The Single Runs archive starts on 2024-03-14, so training starts in March 2024 instead of January 2022. That leaves 13,201 rows before the test year and one winter. Seven days drop out: the archive lacks the 18 UTC run on two and holds empty variables on five. I also rewrote the weather table from scratch.
- **Not working yet.** I start the pipelines by hand. I am setting up Kestra to run features and monitoring daily and training weekly. Inference is a stub, so the retrain trigger scores `champion_eval` instead of logged forecasts. MLflow and the SeaweedFS result store run on my machine, so the executed notebooks in git carry the numbers.
- **Look at first:** `src/winterload/forecast/features/dataset.py` for the run rule and features, `notebooks/backtest/replay.ipynb` for the 08:00 replay, `notebooks/training/feature_study.ipynb` for the comparisons, `src/winterload/repository/` for data access. `make test` runs 129 unit tests without secrets.

2 Response to MS1 reviews

Paths start in `src/winterload/` unless they begin with `notebooks/` or `tests/`. Each entry names its main commit.

B1, also A4 · Target already met · Partly (`bf68dd1`). The target now sits 10% below the preliminary model's validation MAPE: 5.68% sets a bar of 5.12% (`forecast/evaluation/target.py`, MLflow run `preliminary_training`). `forecast/lifecycle/test_year_access.py` refuses a second read of the test year, and its access marker is empty. A test pins the corrected arithmetic, 7.6% to 6.5% as a 14.5% reduction (`tests/forecast/evaluation/test_target.py`). Partly, because the proposal still states the old target (E1).

A1 · Weather archive · Changed (`a521709`). The backfill reads archived single runs. `weather_run` in `forecast/features/dataset.py` picks the 18 UTC run of D-2, or the 12 UTC run before 2024-08-06, when the archive has no 18 UTC cycle. On 28 days against ERA5 its temperature error is 1.28 °C in July and 1.62 °C in February, against 1.34 and 1.48 °C for the 00 UTC run (`notebooks/benchmark/weather_run_choice.ipynb`). To size the gap, I scored the old Historical Forecast source on the same days: 14% less temperature error, and a tie on cloud cover and radiation (`notebooks/benchmark/weather_archive_mape.ipynb`). An earlier 14-day October sample put its advantage at 7 to 22%, so the size depends on the sample. Re-measured MAPE: 5.68% on validation, and 6.75% over six chronological folds against 8.25% for the baseline. B1 rules out a second look at the test year, so the 6.15% gets its successor in the final test.

B2, also A3 · Champion/challenger · Changed (`0152305`). The challenger trains on data up to 8 weeks ago, both models score those 8 weeks, and the winner refits on all data (`pipeline/training/champion_challenger.py`). MLflow keeps two aliases: the refit `champion` has seen the next holdout, and `champion_eval`, which stops at the cutoff, has not. A run on real data before the test year promoted the challenger (5.27% against 5.72%). The first live run can start on 2026-11-18, once the 8-week holdout lies after the test year.

B3, also A5 · Retrain trigger · Changed (`f0791b2`). `forecast/lifecycle/retrain_trigger.py` retrains when the 7-day MAPE exceeds 0.9× the baseline's, and `pipeline/monitoring/retrain_monitor.py` runs it. Evidently will report drift without starting a retrain, and I have not built that report yet. On the week of 2026-10-24 the trigger fired for a model trained in 2025 (8.41% against a threshold of 7.45%). Model against baseline MAPE by season, each month scored by a model trained on earlier rows (`notebooks/backtest/replay.ipynb`): winter 8.78/10.12%, transition 5.66/7.58%, summer 7.61/7.96%. The winter figure rests on 2024/25 alone. An 8-week window sees one part of the year, so the season report scores all 15 months from 2024-07 to 2025-09.

A2 · Clock-change days · Changed (`a521709`). The store keeps UTC, and `local_day_bounds` in `forecast/features/dataset.py` builds local days from the calendar date. Tests pin 23, 24 and 25 hours. The 8,782 hours reconcile: 2025-09-23 to 2026-09-23 holds 8,760 UTC hours, and running on to the last local hour of the end date adds 22. The final MAPE covers 8,760 hours (`tests/forecast/evaluation/test_splits.py`).

A8 · Replayable 08:00 backtest · Changed (`6ae7417`). `notebooks/backtest/replay.ipynb` replays 16 days with the labels and weather known at 08:00 on D-1 and appends the results to logs in the S3 result store. Target hours were complete on all of them: 25 on 2024-10-27, 23 on 2025-03-30, 24 on the rest. Model against baseline MAPE: 9.56/21.22% on 2024-10-27, 3.52/3.80% on 2025-03-30, 5.85/5.49% over a July week and 5.82/7.29% over a January week. The model loses the July week, and the trigger fires there, yet over all summer months the season report keeps it ahead (B3). `pipeline/monitoring/publication_probe.py` logs delays and revisions. Eight observations put Energy-Charts 4.0 to 5.9 h and ENTSO-E 1.6 to 2.5 h behind. The 48 h lag rule tolerates 8 h on the autumn clock change and 9 h on normal days, and neither source revised a value. Open-Meteo published three runs after 6.1 to 7.4 h, so the replay assumes 8 h. I have not yet logged an 18 UTC run. Archive coverage: see A1.

Reviewer A also suggested seasonal models, so I compared a model per season with one model on the same months (`notebooks/training/feature_study.ipynb`, tracked in MLflow). Seasonal against single: winter 5.58/5.72%, transition 6.33/5.66%, summer 7.26/6.73%. The seasonal model wins the two winter months and loses the rest, so I keep one model.

A6 · Holiday definition · Changed (`e683f4e`). `forecast/features/holidays_ch.py` takes each canton's public holidays from the `holidays` package and weights them by population. `holiday_share_ch` is the share of residents with a public holiday on the local date, and `is_holiday_ch` marks a share of at least 50%: New Year, Good Friday, Easter Monday, Ascension, Whit Monday, 1 August, 25 and 26 December. The model trains on the share. MAPE on national holiday hours: share 11.49%, flag 12.08%, no holiday feature 12.45%, baseline 12.08%. Treating national holidays as Sundays scores 10.38% on those hours but 6.81% overall, against 6.75% for the share (`notebooks/training/feature_study.ipynb`). The sample holds 8 holidays, and city order alone moved the flag by 0.5 points.

A7 · Model settings and feature groups · Changed (`4363cee`). `forecast/model/preliminary.py` writes out the settings: 100 trees, learning rate 0.1, 31 leaves, no depth limit, 20 rows per leaf. Early stopping stays off, since it scored 7.24% against 6.75%. On six 8-week chronological folds, all three groups give 6.75%, load alone 7.78%, weather 10.26%, calendar 10.30%, the best pair (load and calendar) 7.73% and the baseline 8.25% (`notebooks/training/feature_study.ipynb`). Shuffling the city order within an hour moves the full model between 6.76 and 6.79%, so I treat gaps below 0.03 points as noise. I kept all three groups. `load_mean_7d` and `heating_degree` from the proposal stay out until a fold test supports them.

B4 · Originality check · Changed (`193b267`). `README.md` lists the projects I checked on mlops-lab.ch (FS26, HS26) and KTH ID2223 (2022/23 to 2026). The closest are gizstrom (daily PV output of one rooftop), NYC electricity demand (daily, trained on observed weather) and Green Energy Prediction (Swedish wind and solar generation). None of them forecasts a country's hourly load. WinterLoad differs on two counts: its live period runs from October into the heating season, with real drift for the trigger, and it trains on forecasts published before 08:00, the input it gets in production.

E1 · Title and header · Not yet. Proposal v2 will carry the title, the HS26 header and the changes above.

E2 · Weather aggregation · Rejected (`e683f4e`). I tested population weights, and they lost: 6.84% against 6.75% for the unweighted mean on the same folds (`notebooks/training/feature_study.ipynb`). I found no demand data by canton, so each canton's population counts towards its nearest city, which gives Zurich 53% and Basel 6%. A comment in `forecast/features/dataset.py` records the choice.